

Compilation Process

1. Preprocessing

- a. Preprocessing is the very first stage of compilation where all lines beginning with a `#` directive are interpreted by the preprocessor.
- b. Examples include `#include`, `#define`, and conditional compilation blocks like `#ifdef` and `#endif`.
- c. The job here is to expand macros, replace header file includes with their actual code, and remove or keep code portions based on conditions.
- d. The output of this phase is typically stored in a `.i` file (pre-processed file). It is still in human-readable C syntax but without directives.

2. Compilation

- a. The `.i` file is passed to the compiler, which translates the pre-processed C code into assembly instructions.
- b. This produces a `.s` file, composed of target architecture-specific assembly code.
- c. In this stage, the compiler also performs syntax and semantic checks—ensuring all rules of the language are followed.
- d. Additionally, optimization may be performed, such as inlining functions or removing redundant instructions, depending on compiler flags (`-O1`, `-O2`, etc.).

3. Assembly

- a. At this stage, the `.s` (assembly) file is handed over to the assembler.
- b. The assembler converts the assembly instructions into machine understandable binary opcodes, forming an object file (`.o`).
- c. `.o` files contain machine code but are not standalone executables since they may still rely on other functions or libraries.
- d. Importantly, symbolic references (like unresolved function calls) are left in place for the linker to handle later.

4. Linking

- a. The linker combines one or more `.o` object files and resolves all symbolic references.
- b. It connects library function calls (like `printf` from the C standard library) and external modules together.
- c. Once all references are resolved and memory addressing finalized, the linker generates a final executable.
- d. Common executable file types: `.exe` (Windows), `.elf` (Linux), `.bin` (bare-metal binary), or `.hex` (firmware/storage-friendly representation of binary).

Cross Compilation

1. List File (“**.lst**”)
 - a. This file contains detailed information about the cross-compilation process.
 - b. It records compiler details, target configuration, and other compilation parameters.
 - c. Has two major parts:
 - i. Page header: Information about the file and compiler.
 - ii. Command line: Compilation commands used by the tool.
 - d. Acts like a log or reference index for how the program was compiled.
2. Preprocessor Output File
 - a. This file contains the expanded result of preprocessing, just like the “.i” file in local compilation.
 - b. Useful mainly for debugging macros, include paths, and conditional directives.
 - c. The extension depends on the cross-compiler in use (not always “.i”).
3. Object File (“**.obj/.o**”)
 - a. Holds the machine code and required records after assembly.
 - b. Contents include:
 - i. Reserved memory details (for global/static variables).
 - ii. Public symbols (variables and function names that can be exported).
 - iii. External symbols (calls to unknown functions).
 - iv. Library links to resolve during final linking.
 - v. Debug information for mapping source lines to binaries.
4. Map File (.map)
 - a. Serves as the linker output providing detailed information about memory layout.
 - b. Contains:
 - i. Page header and build command line.
 - ii. CPU or target architecture details.
 - iii. Input modules and memory arrangement.
 - c. Critical in embedded systems to understand how sections like .text, .data, and .bss are placed in memory.

Executable File Properties

1. .hex File Properties

- a. A .hex file is an ASCII text file encoding binary data as hexadecimal strings, widely used for embedded systems and microcontroller flashing.
- b. The file stores both data and the corresponding memory address, with each line typically describing a segment for loading into target memory.
- c. It contains no symbol or debug info; its structure is designed for ease of transfer and device loading, not human interpretation.
- d. Tools such as flash programmers, converters, or hex editors can interpret and apply .hex files to physical hardware.
- e. The .hex format is highly used for firmware distribution and device programming due to its portability and reliability.

2. .elf File Properties

- a. An .elf (Executable and Linkable Format) file is a common UNIX/Linux binary format containing code, data, symbol, and debug info.
- b. It supports multiple architectures and is flexible for organizing sections like .text (code), .data, .bss (uninitialized data), and more.
- c. Symbol tables and debug info are usually embedded, which supports development, troubleshooting, and dynamic linking.
- d. ELF is readable with tools like readelf, objdump, etc., which show internal structure and symbol details.
- e. The format is used by modern Linux, BSD, and embedded systems, allowing for statically or dynamically linked executables.

3. .bin File Properties

- a. A .bin file is a raw binary image. Unlike .exe or .elf, it contains no relocation, metadata, or symbol/debug info.
- b. It maps directly to memory addresses, useful for embedded systems and microcontroller flashing.
- c. Since there is no symbolic/debugging data, it is not human-readable.
- d. To analyse contents, developers may use tools like hexdump, which print the binary as hexadecimal values.
- e. .bin is extremely efficient for onboard storage and execution, but debugging is harder without extra symbol files.

4. .exe File Properties

- a. An .exe file is a Windows executable containing compiled program code with headers, metadata, and relocation information for running in the Windows environment.
- b. It includes essential sections for the operating system, such as code, data, resources, and import tables for library functions.
- c. Debugging and symbolic information may be included to support development and error tracing.
- d. The format is human-unreadable without specialized tools but is processed by the Windows loader to set up the runtime environment.
- e. .exe files are stand-alone and distributable, but only on compatible Windows systems.